

GALT: Graph-Parallel Augmented-Lagrangian Training with Neural Maps and Safety-Indexed Trace Memory

Yuxuan Liang
vigorfox@live.com

v4 Release
May 2026

Abstract

Backpropagation remains the default training algorithm for modern neural networks, yet it inherits three structural limitations: strict depth-sequential dependence along the chain rule, soft-penalty treatment of external requirements such as safety or retention, and no native mechanism for coordinating graph-structured internal modules. Our earlier CSAT work established that augmented-Lagrangian plus Sherman–Morrison/Woodbury (AL+SM) updates can solve local neural constraints more faithfully than fixed soft penalties, but it also exposed a crucial limit: constraining a pretrained dense carrier is still the wrong representational level. The positive effect remained too entangled with one writable block and too unstable to support a broader paradigm claim.

We introduce **GALT** (Graph-parallel Augmented-Lagrangian Training), a training paradigm that treats a network as a *constraint graph*. Each block is a node, forward consistency is an edge, and external requirements such as safety or memory retention are additional edges in the same optimization object. Training alternates outer augmented-Lagrangian updates with inner parallel block solves. Each block solves a diagonal-plus-low-rank system of the form

$$\left[D_v + \sum_{e \in N(v)} \rho_e J_e^\top J_e \right] \delta_v = -g_v,$$

where Adam’s adaptive denominator provides the diagonal metric and the constraint Jacobians contribute low-rank stiffness terms. Sherman–Morrison and Woodbury identities make these local solves exact in $O(d_v \cdot k^2)$ time for writable subspace dimension d_v and local constraint count k . In the operational sense used in this paper, GALT is a *superset* of the traditional backpropagation regime: when the graph collapses to an ordinary chain and no external constraints are present, it reduces back toward standard first-order training, but when graph structure and persistent constraints matter, it can optimize objects that backpropagation does not natively represent.

The newest algorithmic evidence sharpens this claim further. A GALT update is not a blind local weight change. It requires a *neural map*: a learned or maintained structure that identifies which carriers represent the relevant fact, which parameter sites can express the correction, which safety edges define the admissible cone, how far the local Jacobian remains valid, and which typed channels would be disturbed. This map should be grown during training, not reconstructed only after the fact. Post-hoc mechanistic analysis of an ordinary backprop-trained network may reveal partial circuits, but it does not by itself provide the operational update contract needed for safe structured correction.

The main claim of this paper is not merely algebraic. We develop an empirical bridge from CSAT failure to GALT architecture. Stage C shows that retrofitted coordination on a real Transformer can be made stable but not necessary. Stage D then shows that native routing variables become necessary in toy form, that route-conditioned output experts become necessary on a real carrier once dense fallback is reduced, and that typed task / safety / memory channels survive small multiseed checks. The newest evidence also changes the interpretation of “memory.” Memory should not be treated as a third soft loss or as one more route label. It is better modeled as safety-indexed trace structure: lower admissibility layers quarantine dependent upper memories, higher-layer violations remain locally contained, residual

records unresolved cross-layer debt, and teacher re-adjudication can later convert residual pressure into a structured correction. Symbolic, natural-language, and MLX curriculum diagnostics now support this asymmetry at small scale.

Taken together, the present evidence supports a sharper position than the original CSAT paper could support: GALT is not simply a better constrained optimizer, but the beginning of a training and architecture regime in which task execution, safety behavior, memory retention, and the neural update map itself can become internal, adjudicated, and increasingly necessary structures rather than slack soft-penalty side effects.

1 Introduction

Backpropagation has shaped modern deep learning since Rumelhart–Hinton–Williams [17]. Yet the practical pressures around it keep growing. Larger models intensify sequential depth cost. Safety and alignment objectives are still bolted on as weighted losses. Continual learning remains vulnerable to forgetting, and memory retention is typically handled by replay or fixed penalties rather than by explicit internal organization. The field has become extremely good at engineering around these frictions while rarely questioning the training regime itself.

This paper argues for a different framing. The core problem is not only “optimize a scalar loss faster.” The deeper problem is that the backprop regime has three structural inadequacies for the kind of systems we now want to build:

1. **Sequential-depth dependence.** Computation and credit assignment are chained through depth, which makes the training process fundamentally hostile to graph-parallel execution.
2. **Constraints as second-class objects.** Safety, retention, and other persistent requirements are usually attached as weighted loss terms rather than being treated as things the update rule must actually satisfy.
3. **No native representation of internal responsibility.** Different duties are pushed into the same dense carrier, so the regime has no native way to express task, safety, and memory as separately negotiated internal roles.

In that light, the deeper problem becomes:

How can a model train and update while respecting multiple persistent requirements—task utility, safety behavior, memory retention—without forcing all of them to share one entangled dense carrier?

The refined answer developed in this version is not that task, safety, and memory should simply become three symmetric branches. The more useful structure is hierarchical. Safety should be represented as a stack of constraints with different priorities and override rules. Viability-like constraints should block collapse or immediate danger; harm constraints should block socially inadmissible behavior; operational rules should behave more like soft policies whose violation must be recorded rather than silently forgotten. Memory should consolidate only after this level-aware adjudication. Trace should record which level accepted, rejected, or overrode a candidate transition, and residual should preserve unresolved violations or ambiguities without silently becoming another memory bypass. The newest small-scale evidence makes this more than a design sketch: a safety-indexed hierarchical gate preserves lower memory under higher-layer violations while quarantining upper memory when lower admissibility layers break.

Biological motivation. At a high level, biological nervous systems suggest another useful intuition. Early nervous systems appear comparatively diffuse, whereas later brains depend more heavily on differentiated yet still coordinated structures. Memory, threat evaluation, and goal-directed control are not all forced through one homogeneous substrate; they are carried disproportionately by partially specialized systems that remain richly interconnected. The lesson is not that artificial systems should literally copy neuroanatomy. It is that sustained competence under competing demands seems to benefit from some degree of internal responsibility separation rather than one undifferentiated carrier.

This intuition is also broadly consistent with behavioral neuroscience. Reviews of stress–memory interactions report that uncontrollable stress generally impairs hippocampal-dependent memory and hippocampal function [9, 10]. Conversely, the safety-learning literature treats learned safety signals as cues that predict the nonoccurrence of threat, inhibit fear and stress responses, and form a distinct learned memory rather than merely suppressing output [18]. Recent human work further shows that safety signal learning reduces threat responding and involves hippocampal activation and hippocampal-cingulate connectivity [15]. We do not use these literatures as proof of a shared mechanism with GALT. We use them more narrowly as convergent evidence that safety / boundary structure can play an organizing role for memory and regulation, not only a late veto role.

Modern foundation models still look closer to the diffuse end of that spectrum. Task performance, safety alignment, and memory retention are largely entangled inside shared parameters and shared representations. Safety is usually imposed through external fine-tuning procedures or auxiliary objectives rather than through native internal pathways. This design is powerful, but it also means that the same dense carrier is asked to absorb new skills, preserve old knowledge, and remain safe under update.

From that perspective, GALT is partly biologically inspired. Its goal is not to mimic hippocampus, amygdala, or prefrontal cortex in any literal sense. Rather, it aims to move toward architectures in which different persistent responsibilities can begin to occupy distinguishable internal channels while still coordinating inside one system. The typed task / safety / memory channels explored here are only an initial step, but they point in that broad direction: from diffuse integration toward structured specialization without giving up integrated behavior.

Our earlier work, **CSAT** (Constrained Step-wise Augmented-Lagrangian Training), attacked this question at the optimizer level. Its basic idea was simple: keep the ordinary task update as the base motion, then insert a local augmented-Lagrangian projection in the writable subspace so that safety or retention constraints are enforced by the update rule itself rather than only by a weighted loss. This established a real result: solving constraints is materially different from merely penalizing them. However, CSAT also exposed its own limit. Even with better local constrained updates, a single dense writable carrier remains too entangled. The optimization algebra improves, but the representation regime does not fundamentally change.

The newer **GALT** line starts from that failure and changes the object of study. Instead of asking how to constrain one dense carrier more cleverly, it asks how to train *graphs of local blocks connected by explicit protocol variables*. In that picture:

- forward consistency is not implicit in the chain rule; it becomes an explicit constraint;
- safety and memory are not mere auxiliary penalties; they become additional edges in the same graph;
- routing is not just a sparse efficiency trick; it becomes a candidate internal protocol variable that can be tested for causal necessity.

This change in framing matters for *sustainable learning*. By “sustainable learning” we mean more than conventional continual learning. We mean a training regime in which new learning can occur without immediately collapsing safety, retention, or other persistent requirements. The intended mechanism is not simply “add a better regularizer,” but move those requirements into the optimization object and, increasingly, into internal channels rather than one overloaded dense path.

Main claim. GALT currently supports a stronger sustainable-learning claim than CSAT did, because the positive effect is beginning to appear as a repeatable *architectural phenomenon*, not only as a single-block optimizer-side projection.

Paper contribution. This paper replaces the older CSAT-first narrative with a new GALT-first manuscript. Its role is to:

1. explain why CSAT was a necessary precursor but not the final frame;
2. formalize GALT as graph-parallel augmented-Lagrangian training;

3. present the empirical bridge from Stage C failure to Stage D necessity;
4. show the chart-local BP-inside-GALT correction limit and the neural-map requirement for nonlinear update sites;
5. introduce *safety-indexed trace memory* as the current architectural derivative of map-guided admissible correction.

What this paper does not claim. We do not claim that GALT is already a production-ready replacement for backpropagation, nor that typed channels are fully disentangled, nor that plugin-like loading is already achieved. The present contribution is narrower and more important: a new route has become experimentally visible, repeatable enough to take seriously, and more compelling than the older CSAT path.

2 Why CSAT Was Necessary but Not Sufficient

2.1 What CSAT Actually Established

CSAT established a real algebraic point. In settings where the writable subspace is small, a constrained update of the form

$$\delta = d_{\text{adam}} - \frac{f_c + \rho\tau}{1 + \rho\alpha} \tilde{J}$$

can be computed exactly with Sherman–Morrison or Woodbury corrections on top of Adam’s diagonal metric. In practical terms, this means:

- soft penalties are not the only way to incorporate constraints;
- augmented-Lagrangian tracking avoids the worst penalty-method pathology described by Bertsekas [3];
- the AVBD-style $\rho J^\top J$ stiffness pattern from physics is genuinely transferable at the algebraic level [2].

This was already valuable. It ruled out the idea that neural constraints must always be implemented as weak weighted loss terms.

CSAT also produced enough empirical evidence to matter as more than a purely algebraic observation. On dense Split-MNIST, the full AVBD-style optimizer reached 0.964 accuracy with 0.037 forgetting while performing roughly 90% local steps, showing that local constrained solves can work strongly in a favorable small-scale regime. On the AG News Safety-from-Birth line, the Sherman–Morrison Hessian variant reached 0.434 task accuracy across a 4-task 3-seed study, slightly above a fixed KL-sum baseline at 0.423 and well above a post-hoc AVBD baseline at 0.266, showing that AL-based local projection can be competitive in LLM continual fine-tuning. On the GPQA bridge, however, the margin over Adam/EWC remained narrow (0.259 versus 0.252), which is exactly the warning sign that optimizer-side improvement alone is not yet a new architecture-level regime.

2.2 Why CSAT Could Not Carry the Main Paradigm Claim

The problem was not that CSAT was “wrong” in a narrow sense. The problem was that it remained trapped at the wrong representational level.

1. **Single dense carrier.** The main writable path was still one dense carrier, so different duties remained entangled.
2. **No native protocol variable.** The model did not expose an internal low-dimensional route or protocol that downstream operators had to consume.
3. **No causal channel claim.** Positive outcomes could still be read as improvements in local constrained optimization rather than as evidence of a new internal organization regime.

In hindsight, CSAT should be treated as a *historical precursor and negative boundary*. It exposed the AL+SM core and the failure of fixed soft penalties, but it did not yet establish a path toward responsibility-separated internal channels.

3 The GALT Paradigm

3.1 Constraint Graph View

GALT reframes training as constraint satisfaction on a graph:

$$G = (V, E_{\text{fwd}} \cup E_{\text{ext}}),$$

where each computational block is a node, forward consistency relations are architectural edges, and task-external requirements such as safety or memory are additional edges. In the simplest form:

$$\begin{aligned} V &= \{\text{Block}_v : v = 1, \dots, L\}, \\ E_{\text{fwd}} &= \{(v, v+1)\}, \\ E_{\text{ext}} &= \{(v, *) : \text{safety} / \text{retention} / \text{fairness} / \text{policy}\}. \end{aligned}$$

Each block v holds a local parameter subspace $\theta_v \in \mathbb{R}^{d_v}$ and an Adam-derived diagonal metric D_v . Each edge e contributes a constraint value C_e , multiplier λ_e , penalty ρ_e , and Jacobian J_e .

This graph is not sufficient by itself. A neural implementation also needs a local *map* M_v for each candidate update site. The map records, or computes on demand, which typed carriers are relevant, which constraints touch the site, whether the target correction is reachable through the local Jacobian, how stable the current linearization is, and which other channels may be disturbed by the update. In ordinary back-propagation these relations are implicit in a dense gradient field. In GALT they become part of the object being trained.

3.2 Local Block Solve

Fix all neighbors of block v . The local constrained subproblem is

$$\min_{\theta_v} f_{\text{local}}(\theta_v) + \sum_{e \in N(v)} \left[\lambda_e C_e(\theta_v) + \frac{\rho_e}{2} C_e(\theta_v)^2 \right].$$

Linearizing each C_e around the current point produces the local system

$$\left[D_v + \sum_{e \in N(v)} \rho_e J_e^\top J_e \right] \delta_v = -g_v.$$

The matrix is diagonal-plus-low-rank. With one active local constraint, Sherman–Morrison gives an exact $O(d_v)$ solve; with k local constraints, Woodbury gives $O(d_v \cdot k + k^3)$ complexity. For small k , the dominant term is effectively linear in the writable dimension.

3.3 Outer and Inner Iteration

GALT alternates:

1. a **map** phase, where candidate update sites estimate carrier relevance, Jacobian reachability, admissibility, local stability radius, and typed-channel interference;
2. an **inner** parallel local-update phase, where blocks solve their own subproblems while exchanging messages with neighbors;
3. an **outer** augmented-Lagrangian phase, where multipliers are updated and penalties adapt to persistent violation.

Algorithm 1 GALT: Graph-Parallel Augmented-Lagrangian Training

```

1: for outer iteration  $k = 1, \dots, K_{\text{out}}$  do
2:   build or refresh neural maps  $M_v$  for candidate update sites
3:   for inner iteration  $j = 1, \dots, K_{\text{in}}$  do
4:     for all blocks  $v \in V$  in parallel do
5:       score candidate sites by reachability, admissibility, stability, and interference
6:       receive local messages  $(\lambda_e, \rho_e, C_e, J_e)$  from neighbors
7:       form  $H_v = D_v + \sum_{e \in N(v)} \rho_e J_e^\top J_e$ 
8:       form local right-hand side  $g_v$ 
9:       solve  $H_v \delta_v = -g_v$  via SM/Woodbury
10:      update  $\theta_v \leftarrow \theta_v + \eta \delta_v$  inside the trusted map region
11:      refresh affected constraints and send updated messages
12:    end for
13:  end for
14:  for all external edges  $e \in E_{\text{ext}}$  do
15:     $\lambda_e \leftarrow \max(0, \lambda_e + \rho_e C_e)$ 
16:     $\rho_e \leftarrow \text{adaptive\_update}(\rho_e)$ 
17:  end for
18: end for

```

3.4 The Four Pillars

GALT depends on four inseparable pillars:

Pillar	Role	Without it
Graph decomposition	exposes local blocks and message edges	reduces back toward a monolithic chain
ADMM-style splitting	makes local subproblems independently solvable	requires a global coupled solve
Augmented-Lagrangian tracking	removes finite-penalty bias	collapses back to soft-penalty behavior
Local SM/Woodbury solve	makes constrained local updates exact and cheap	leaves only slow first-order correction

The conceptual target is not only parallelism. It is the emergence of **adjudicated responsibility-separated channels**: internal routed paths whose duties are not merely named, but ordered. Task proposes behavior, safety defines an ordered constraint stack, memory consolidates trace-bearing structure only after level-aware adjudication, and residuals carry unresolved constraint debt forward.

3.5 Neural Maps as Update Contracts

The local system above is only meaningful when the solver knows where a correction can be expressed. We call this structure a *neural map*. It is not merely an interpretability diagram of a completed model. It is an operational update contract:

$$M_v = \{\text{carrier relevance, Jacobian reachability, admissibility cone, stability radius, conditioning, typed-channel interference}\}.$$

Given examples and constraints, M_v answers which site should be updated, whether the desired correction is locally reachable, whether the safety stack admits the update, and how far the current linear approximation can be trusted.

This distinction is important. Open weights expose a model’s behavior, but not necessarily the update map required to modify that behavior safely and selectively. A post-hoc circuit analysis may discover partial features or attribution paths, yet still fail to recover the full admissibility and low-interference update structure. GALT therefore treats neural-map formation as part of from-scratch training. The goal is not to train an ordinary dense network and later reverse-engineer where to intervene; it is to make carrier ownership, local reachability, and admissible correction geometry develop together with the model.

The first neural-layer diagnostic supports this ordering. When the update site is the output head, logit-margin constraints are exactly linear in the head weights, so the local map is complete and a one-step GALT solve succeeds. When the update site is a ReLU hidden adapter, a raw KKT step can leave the current activation chart and invalidate the Jacobian. A boundary-only activation map prevents this failure but becomes too conservative. The next algorithmic target is therefore map-guided site selection: choose the carrier or module whose local chart can express the desired correction with admissible safety and low interference, then solve only inside that trusted region.

3.6 Stratified Safety and Adjudicated Trace Memory

The revised GALT substrate treats memory as a constrained transition system rather than as a third objective term. For a proposed update or candidate trace z_t , safety is not modeled as a single centralized boundary. Instead it is a stratified constraint stack:

$$C_0(z_t) \leq 0, \quad C_1(z_t) \leq 0, \quad C_2(z_t) \leq 0, \quad C_3(z_t) \leq 0.$$

The intended semantics are ordered. C_0 captures hard admissibility such as collapse risk or immediate danger; C_1 captures social, consent, and harm boundaries; C_2 captures task or workflow rules; C_3 captures durable preferences and other higher-level memories. The important point is not the exact naming of four levels, but the dependency ordering: lower layers define the feasible region in which higher memories may be consolidated and later accessed.

For a compact implementation, the hard admissibility condition is

$$C_{\text{hard}}(z_t) = \max(C_0(z_t), C_1(z_t)) \leq 0,$$

with higher layers contributing to residual state rather than acting as absolute global erasers. An adjudication gate $a_t \in [0, 1]$ then controls consolidation:

$$m_{t+1} = m_t + a_t \text{Consolidate}(z_t),$$

where a_t should be high only when the candidate passes the lower-level hard safety stack. The trace state records the adjudicated transition and the level that accepted, rejected, or overrode the candidate,

$$\tau_{t+1} = \text{Trace}(\tau_t, z_t, a_t, C_0(z_t), C_1(z_t), C_2(z_t), C_3(z_t)),$$

and the residual state carries unresolved violation, ambiguity, or soft-rule override debt,

$$r_{t+1} = \text{Residual}(r_t, z_t, C_0(z_t), C_1(z_t), C_2(z_t), C_3(z_t), a_t).$$

The resulting memory ledger is safety-indexed. If a memory trace depends on layer m and the current active violation occurs at layer v , access follows the asymmetric rule

$$A(m, v) = \begin{cases} \text{access}, & v = \emptyset, \\ \text{quarantine}, & v \leq m, \\ \text{access}, & v > m. \end{cases}$$

Thus a lower-layer violation invalidates or quarantines dependent upper memory, while a higher-layer violation remains locally contained and should not destroy lower capability or lower-layer memory. This is the key difference between hierarchical GALT memory and both same-layer-only gates and global flush gates.

This distinction matters for the paper’s central claim. A memory branch that merely receives a retain loss is still a soft-penalty mechanism. A memory edge that can only consolidate after level-aware safety

adjudication is a constraint-native mechanism. The current experiments now provide a first small-scale mechanism chain for this substrate: symbolic layer gates expose the needed bottom-up invalidation and top-down containment asymmetry, natural-language contrast packets preserve the same asymmetry, and the MLX curriculum model learns memory-layer and violation-layer heads whose predicted hierarchical access gate reaches perfect held-out access accuracy in the 600-step violation-weighted smoke. This is not yet a scaled language-model result, but it is no longer only a paper hypothesis.

3.7 Why GALT Is a Superset of Backpropagation

The word “superset” should be read carefully. We do not claim a formal theorem that every practical backprop training recipe is already recovered in finished GALT form. We make a narrower but important claim: GALT contains the *kind of optimization object* used by backprop as a special limiting case, while also extending beyond it.

1. **If the graph collapses to a simple chain**, GALT reduces toward the ordinary layered setting that backprop assumes.
2. **If external edges are removed**, GALT reduces toward pure task-loss training rather than joint task-plus-constraint negotiation.
3. **If the low-rank constraint terms vanish**, the local block solve reduces toward an ordinary first-order update under the base diagonal metric.

Backprop therefore sits inside GALT as the regime where there is only one effective objective, one chain-structured dependency pattern, and no explicit constraint-native protocol variables. GALT becomes strictly richer when any of those assumptions are relaxed. It can represent graph-structured message passing, persistent external constraints, and internally testable routed channels in one common optimization frame. That is the sense in which it is best understood as a superset rather than merely as an alternative optimizer. The newest unified correction validation now gives a first empirical version of this statement with an important qualification. On a deliberately chart-matched output-only regime, single-site GALT and an output-only iterative BP limit become effectively identical across a three-seed check, whereas on constraint-active cases full GALT remains much stronger than reduced GALT, single-site GALT, and both BP-like limit modes. This supports *chart-local* BP-inside-GALT, but it does not yet prove that the full general training regime of practical backpropagation has already been recovered inside mature GALT form. A first token-bearing student on the same shared execution family preserves this qualitative split as well, which suggests that the result is not tied only to direct dense descriptor input.

4 Empirical Bridge from CSAT Failure to GALT

4.1 Stage C: Stable Coordination Is Not Enough

The first bridge stage asked whether a retrofitted narrow coordination stream on a real Transformer could become a genuine internal carrier. The answer was negative. Deep supervision, late-heavy supervision, and future-summary control objects could improve the coordination readout, but the downstream *necessity* gaps remained near zero. In plain language: the model could carry a coordination signal, but it did not need to.

This result matters because it rules out a tempting false positive. Merely observing a clean narrow latent is not sufficient evidence of a new regime.

4.2 Stage D Toy: Native Routing Can Be Necessary

The next step changed the object being trained. Instead of a retrofitted summary-like stream, the model received an explicit routing / policy variable consumed by downstream experts. In toy form, this immediately produced strong necessity: zeroing or scrambling the routing variable caused large performance drops. This established that the missing ingredient was not “more supervision” but a *native decision-bearing protocol structure*.

Table 1: Empirical bridge from CSAT to GALT. The main trend is not simply better numbers, but increasing evidence that the constrained path becomes architecturally necessary.

Stage	Setting	What it established
CSAT	single dense writable carrier	AL+SM core is real, but architecture still too entangled
Stage C	retrofit dual-stream carrier on Qwen	coordination can be stable without becoming necessary
Stage D toy	native routing toy	explicit routing variable can become necessary immediately
Stage D real carrier	routed output experts + fallback reduction	zero-gap and scramble-gap become strongly positive on a live Transformer
Typed Stage D	task / safety / memory branches	typed routed channels survive small multiseed checks; safety appears to scaffold memory more reliably than memory-only routing
Stage D/E kernel	safety-indexed hierarchical trace memory	lower-layer violations quarantine dependent upper traces; higher-layer violations are locally contained; residual stores unresolved cross-layer debt

4.3 Stage D Real Carrier: Dense Fallback Was the Main Obstacle

On a real Qwen-MLX carrier, the first routed hidden-adaptor prototype made route learning possible but not necessary. The dense pretrained output path still dominated. The first real-carrier breakthrough came only after moving the routed effect into *task-bearing output experts* and explicitly reducing dense fallback in final choice scoring.

4.4 Typed Channels, Anti-Shadowing, and the Need for Adjudication

Once the real-carrier route became necessary, the next question was whether it could support distinct typed duties. The answer is now cautiously positive.

Typed baseline. With expert-only outputs and typed task / safety / memory branches, the strong single-seed regime reaches:

$$(\text{task}, \text{safety}, \text{retain}) = (0.875, 0.875, 0.9),$$

with positive task zero-gap and scramble-gap, and with safety also showing positive route dependence. This already goes beyond the CSAT regime, because the effect now begins to live in typed architecture rather than in one local projection.

Small multiseed check. Across three seeds, the typed baseline yields:

$$\text{post task} = 0.8125 \pm 0.088, \quad \text{post safety} = 0.8333 \pm 0.059, \quad \text{post retain} = 0.7000 \pm 0.141.$$

This is not fully mature, but it is no longer a one-seed accident.

Safety anti-shadowing. Moderate safety-only anti-shadowing does not materially raise the mean utility, but it improves the safety branch margin from a slightly negative mean to a positive one in the small multiseed check. This makes its role clear: it is best understood as a *branch-boundary sharpening mechanism*, not as a raw benchmark booster.

Memory remains the weaker frontier. The memory / retain channel remains more seed-sensitive and less cleanly separated. Memory-only anti-shadowing has not yet moved the current regime. This is an honest limitation and also a useful result: responsibility-separated channels are now visible, but not yet complete. It also suggests that memory should not be modeled as an independent branch that competes symmetrically with task and safety. The stronger interpretation is that memory needs an adjudication mechanism: a candidate trace should become persistent only when it is admissible under the safety edge.

Safety as a candidate memory scaffold. The newer Stage D controls sharpen the interpretation of that limitation. In the typed routed regime, safety-route supervision repeatedly organizes retain behavior more effectively than memory-route-only supervision. On a pure counterfactual retain benchmark, the corrected safety-route setting yields retain branch accuracies

$$(\text{task}, \text{safety}, \text{memory}) = (0.1667, 0.5000, 0.6667),$$

with a positive retain zero-gap of 0.3333. Adding direct memory-route supervision does not make route scrambling harmful, but it does further sharpen retain exclusivity to

$$(\text{task}, \text{safety}, \text{memory}) = (0.0000, 0.3333, 0.6667),$$

raising the preferred-branch margin over the best wrong branch to +0.3333. The cleanest current reading is therefore asymmetric: the routed carrier already matters for memory, but the model is using a structured safety-like route scaffold more than a uniquely identified memory route code.

Five-seed route-identity extension. The stronger V2-oriented robustness check keeps the same counterfactual retain benchmark and compares the plain scaffold baseline against typed route policies plus a fixed memory-route identity anchor. Across five seeds, the scaffold baseline gives a mean retain scramble-gap of only

$$0.0333 \pm 0.2211,$$

with positive scramble-gap in only two of five seeds. Under typed route policies with the same counterfactual benchmark, mean retain accuracy rises to

$$0.8000 \pm 0.1247,$$

mean retain zero-gap and scramble-gap both rise to

$$0.2333 \pm 0.1700,$$

and positive retain scramble-gap appears in four of five seeds. This is still not industrial robustness, and one seed falls into a bad specialization regime with low task / safety utility. But it is now strong enough to support a more disciplined claim: typed route policies increase the probability that a counterfactual memory channel acquires a *shallow* route identity.

Crucially, the localization diagnosis does not change. In the same-seed causal ladder, scaffold alone creates memory ownership and route presence, direct memory-route supervision sharpens ownership without making scrambling harmful, fixed identity anchors make retain scramble-sensitive, and typed route policies amplify that effect further. Yet the split-scramble controls still show that scrambling block-internal route assignments is largely harmless while scrambling the route-conditioned output identity reproduces the retain drop. The current positive identity effect therefore lives mostly at the output head, not yet in deep block-carrier routing.

This is why we treat the safety result as more than an anti-shadowing curiosity. Taken together with the behavioral literature on learned safety signals and stress-sensitive hippocampal memory [18, 9, 10, 15], the present bridge supports an important candidate design principle: in high-responsibility learning systems, safety may function not only as an output guardrail but as a boundary-setting substrate on which more specific memory specialization grows. In this paper that remains a theoretical claim supported by convergent evidence, not a theorem and not a literal biological identity.

Table 2: Safety-indexed memory evidence chain. The reported numbers are small-scale diagnostics, but they show the same bottom-up invalidation / top-down containment asymmetry across symbolic, natural-language, and MLX curriculum settings.

Diagnostic	Main comparison	Key result
Symbolic layer gate	independent same-layer vs global flush vs hierarchical	hierarchical gate gets bottom-up and top-down heldout accuracy 1.000; independent fails bottom-up; global flush fails top-down
Natural-language contrast gate	flat text-only learners vs structural gates	balanced BoW partially learns the rule, but hierarchical gate preserves both asymmetries at 1.000
MLX curriculum layer heads	trainable memory/violation layer heads plus predicted access	600-step violation-weighted recipe passes seeds 42/43/44: mean action/gate/memory 1.000, hard-violation recall 1.000, predicted hierarchical access 1.000

Adjudicated memory hypothesis. The claim is therefore sharper than “memory has a route.” It is: memory becomes stable when it is trace-bearing structure consolidated inside a stratified safety stack. In this formulation, lower-level viability and harm edges define hard admissibility, upper operational rules define soft override debt, the memory edge performs consolidation, the trace records accepted and rejected transitions, and the residual stores unresolved violations, ambiguities, or soft-rule violations. This explains why the current 7B retain branch is the unstable part of the system: Stage D has begun to expose trace-bearing memory, but deep route identity and residual-resolution remain immature.

4.5 Safety-Indexed Hierarchical Memory Evidence

The latest experiments test the stronger version of the memory story:

A lower-layer safety violation should quarantine dependent upper memory, but a higher-layer rule violation should not destroy lower-layer memory.

This asymmetry is not captured by two simpler designs. A same-layer-only gate protects lower memories from higher violations, but misses lower-to-upper invalidation. A global flush catches lower-to-upper invalidation, but destroys lower memory whenever any higher rule is violated. A hierarchical gate keeps both properties by indexing each memory trace by its dependency layer.

The MLX smoke uses a compact generated curriculum, not a large language model: 68 accepted contrast packets are expanded into adjudicated memory rows, with packet-balanced sampling and explicit memory-layer, violation-layer, and hierarchical-access targets. Its residual accuracy is 0.969 on the held-out test split, with boundary-ambiguity recall 0.900 and hard-violation recall 1.000 in the first seed. A follow-up three-seed check of the same violation-weighted 600-step recipe preserves the result: action, gate, memory, accepted-trace, not-consolidated, hard-violation recall, and predicted hierarchical access all have mean 1.000, while residual accuracy has mean 0.984 and violation-layer accuracy has mean 0.976. The important methodological point is that the same heads learn accept/reject, memory/not-consolidated, residual type, memory layer, violation layer, and predicted access together. In this small setting, longer GALT-stage training and violation-layer class weighting close the previous interference gap rather than collapsing consolidation.

This result upgrades the stratified-memory claim from a next-step target to a small-scale proof of mechanism. It should still be reported conservatively: the strongest gate result is now a three-seed compact generated-curriculum check, not a large language-model memory system. The next robustness test is a larger contrast lesson set and integration into a broader language-bearing student.

5 Why GALT Better Supports Sustainable Learning

We can now return to the original question of sustainable learning. Compared with CSAT, GALT supports a stronger claim for three reasons.

5.1 It Moves from Constraint Patching to Internal Organization

CSAT mainly improved how one writable carrier is updated. GALT begins to change where different duties live. This matters because sustainable learning is not only about constraining updates; it is about keeping task progress, safety behavior, and memory retention from destructively overwriting one another.

5.2 It Demands Necessity, Not Merely Presence

The GALT line explicitly tests whether a routed or typed channel is *necessary* by zeroing or scrambling it. That is much stronger than showing that some latent representation or auxiliary head correlates with a target. The sustainable-learning claim becomes architectural only when the channel is causally operative.

5.3 It Opens the Door to Responsibility-Separated Channels

If task, safety, and memory can become internal typed channels, then continual adaptation no longer has to be phrased as “keep one dense model from forgetting everything.” It can begin to be phrased as:

update one part of the system while maintaining negotiated consistency across several internal responsibilities.

That is why GALT is a better candidate for sustainable learning than the older CSAT framing, even though the present implementation is still incomplete.

5.4 It Turns Memory into an Adjudicated Transition

The newer typed controls support a stronger claim than “safety and memory can sometimes be separated.” They suggest an *ordering*: a safety-like boundary channel may become useful before a fully distinct memory-identity channel is learned. In the revised substrate, memory is not the act of minimizing retain loss. It is the act of accepting a proposed trace as persistent structure after safety adjudication. This makes the training object more precise:

stratified safety stack $\rightarrow$ level-aware admissibility, memory edge $\rightarrow$ adjudicated consolidation.

Trace records which transitions were accepted or rejected, while residuals store hard safety violations, branch disagreements, soft-rule overrides, or ambiguous retain events. In our current Stage D regime, retain behavior already depends on routed structure, and memory branch ownership can become clean on a counterfactual retain set. In the scaffold baseline, retain scramble-gap often remains zero even when retain zero-gap is positive; with fixed identity anchors and typed route policies, positive retain scramble-gap becomes achievable and even repeatable across a small five-seed probe, but the effect still localizes primarily to the output head. The current pattern is therefore more precise than before: route presence matters before route identity matters, and shallow route identity appears before deep route identity. The missing mechanism is explicit adjudication and residual resolution. The newest compact curriculum experiments add the first version of that missing mechanism: learned memory-layer and violation-layer heads drive a hierarchical access gate that keeps lower memory available under higher-layer violations while quarantining dependent upper memory under lower-layer violations.

This ordering is notable because it converges with external literatures in which safety signals, controllability, and predictable non-threat conditions help buffer stress responses, while uncontrollable stress degrades hippocampal-dependent memory [18, 9, 10]. GALT does not yet prove that this is a universal law of intelligence. But it does make the hypothesis experimentally respectable: safety may be a low-level organizational substrate that helps stabilize later memory specialization, rather than merely a late-stage refusal head attached to an otherwise complete system.

6 Relation to Existing Work

GALT intersects several lines of work but is not reducible to any one of them.

Backpropagation and its variants. Backpropagation [17] and modern first-order optimizers such as Adam [8] remain the dominant training regime. Natural gradient and curvature approximations improve local conditioning but still live inside the backprop frame [1, 5, 14].

Continual-learning penalties. EWC and synaptic-intelligence style methods penalize motion in directions deemed important for earlier tasks [11, 21]. They are valuable baselines, but they remain soft-penalty treatments of the problem.

Alternative local-credit ideas. Forward-forward and target-propagation style work challenge standard backprop assumptions [6, 12]. GALT shares the desire to loosen strict chain dependence, but differs by explicitly casting training as augmented-Lagrangian constraint satisfaction on a graph.

Physics-derived local solves. The most direct algebraic ancestor of GALT is AVBD-style block-local augmented-Lagrangian physics solving [2]. What transfers is not the five-step solver checklist, but the invariant that a diagonal base metric plus low-rank $\rho J^\top J$ terms gives locally exact, constraint-aware updates.

Distributed and graph-native training. Work such as DiLoCo shows that relaxing communication structure can matter for large-scale training [4]. GALT differs by treating message passing and external constraints as part of one common optimization object rather than as a communication wrapper around standard backprop.

7 Limitations and Next Steps

The present paper is strongest as a bridge paper, not as a final solved-paradigm paper.

1. **Typed channels are real but still shallow.** Safety is more controllable than memory; retain remains the weaker and more seed-sensitive path on the 7B carrier. The compact curriculum now has a working level-aware safety gate, but the deeper real-carrier route identity is still primarily an output-head effect.
2. **The multiseed checks are still modest.** Five seeds on the new counterfactual shallow-identity line and three seeds on the older typed baseline are enough to rule out pure accident, but not enough to establish stable mature routing.
3. **Stratified safety is proven only at small scale.** The symbolic, natural-language, and MLX curriculum diagnostics support the hierarchical memory rule, but the strongest trainable gate is still one seed on a compact generated curriculum. It needs multiseed and larger lesson support before it can be treated as robust.
4. **Plugin-like loading is not yet justified.** Responsibility-separated channels are a prerequisite, not yet the final plugin system.
5. **Full graph-scale GALT is future work.** The present positive evidence is still on targeted Stage C / Stage D prototypes rather than on a full many-block end-to-end graph deployment.

The immediate paper-oriented next steps are therefore clear:

- strengthen the empirical bridge rather than over-optimize one narrow subproblem;
- use typed multiseed repeatability and anti-shadowing as evidence of responsibility-separated channels;
- scale the safety-indexed hierarchical memory gate beyond the current compact generated curriculum;

- test whether residual debt can be re-adjudicated into useful corrections without collapsing accepted-trace consolidation.

8 Conclusion

CSAT showed that solving constraints is different from merely penalizing them, but it could not escape the dense-carrier regime. GALT begins where CSAT stops. It treats training as graph-structured augmented-Lagrangian constraint satisfaction, uses local Sherman–Morrison/Woodbury solves on top of Adam’s metric, and, most importantly, starts to move useful duties into internal routed channels.

The strongest current claim is therefore not that GALT is already the final replacement for backpropagation. It is that a new route has become visible:

native protocol variables, task-bearing routed experts, reduced dense fallback, and typed responsibility-separated channels can begin to produce repeatable architectural effects, while memory becomes most coherent when it is an adjudicated, safety indexed trace ledger rather than a symmetric retain branch.

That is already a stronger and more defensible position than the older CSAT paper could support. If sustained, it points toward a training regime in which sustainable learning is supported not only by constrained updates, but by the internal organization of the model itself. One especially important candidate principle emerging from the present bridge is that safety-like boundary channels may scaffold memory formation and retention rather than merely policing outputs: safety supplies level-aware admissibility, lower-layer violations quarantine dependent upper traces, higher-layer violations are contained locally, and residuals carry unresolved violations or soft overrides forward instead of letting them silently become hidden memory.

The newest superset-validation result sharpens that conclusion. The current evidence is now strong enough to support the narrower claim that *chart-local BP-like iteration appears as a degenerate execution mode inside GALT*: on a matched output chart, output-only iterative BP and single-site GALT coincide almost exactly, and this remains true across a small three-seed robustness check. But the same experiment also shows why the broader GALT frame matters: once hidden-site routing, teacher fallback, and constraint-aware admissibility become structurally necessary, full GALT retains a large and stable advantage. The remaining open step is therefore no longer “can BP ever appear inside GALT?” but “can this local equivalence be lifted into a broader end-to-end training regime?” The newest token-bearing student result modestly strengthens that bridge: the same qualitative pattern survives when the controller consumes a discrete token sequence rather than a direct dense descriptor vector, although this is still far from a finished language-bearing GALT-native learner. The newest hierarchical-memory result adds a complementary derivative: once correction is gated by admissibility, memory access itself should be indexed by the safety layer on which the trace depends.

References

- [1] S. Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.
- [2] C. Giles, E. Diaz, and C. Yuksel. Augmented vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2025)*, 44(4), 2025.
- [3] D. P. Bertsekas. *Nonlinear Programming*. Athena Scientific, 2nd edition, 1999.
- [4] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, and A. Szlam. DiLoCo: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023.
- [5] V. Gupta, T. Koren, and Y. Singer. Shampoo: Preconditioned stochastic tensor optimization. In *Proceedings of ICML*, pages 1842–1850, 2018.
- [6] G. E. Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

- [7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*, 2022.
- [8] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of ICLR*, 2015.
- [9] J. J. Kim and D. M. Diamond. The stressed hippocampus, synaptic plasticity, and lost memories. *Nature Reviews Neuroscience*, 3(6):453–462, 2002.
- [10] E. J. Kim, B. Pellman, and J. J. Kim. Stress effects on the hippocampus: A critical review. *Learning & Memory*, 22(9):411–416, 2015.
- [11] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [12] D.-H. Lee, S. Zhang, A. Fischer, and Y. Bengio. Difference target propagation. In *Proceedings of ECML PKDD*, pages 498–515, 2015.
- [13] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *Proceedings of ICLR*, 2019.
- [14] J. Martens and R. Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *Proceedings of ICML*, pages 2408–2417, 2015.
- [15] P. Odriozola, S. Kribakaran, E. M. Cohodes, et al. Hippocampal involvement in safety signal learning varies with anxiety among healthy adults. *Biological Psychiatry Global Open Science*, 4(1):155–164, 2024.
- [16] B. T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [17] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [18] J. P. Christianson et al. Inhibition of fear by learned safety signals: A mini-symposium review. *The Journal of Neuroscience*, 32, 2012.
- [19] P. Tseng. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of Optimization Theory and Applications*, 109(3):475–494, 2001.
- [20] S. J. Wright. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34, 2015.
- [21] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *Proceedings of ICML*, pages 3987–3995, 2017.